n-step 부트스트래핑, 적격흔적, 그리고 계획

n-step TD · 적격흔적(TD(λ)) · 계획과 학습: Dyna-Q

Machine Learning 2 · 7주차

한국과학영재학교 (KSA)

Week 7

이번 주차 개요

- **7.1 n-step TD** — MC와 TD(0) 사이의 다이얼, U자 곡선
- **7.2 적격흔적** — TD(λ)로 모든 상태를 동시에 갱신
- **7.3 Dyna-Q** — 모델 저장 + 가상 갱신으로 약 9.5배 가속

이전·다음과의 연결

n-step TD, TD(λ), Dyna-Q는 모두 **"부트스트래핑을 얼마나 할 것인가"**라는 하나의 질문에 대한 서로 다른 답. Week 8 캡스톤에서 Gymnasium의 표 기반 환경으로 세 개를 직접 구현·비교하고, n-step 리턴은 Week 9 DQN의 기본 옵션으로 다시 만난다.

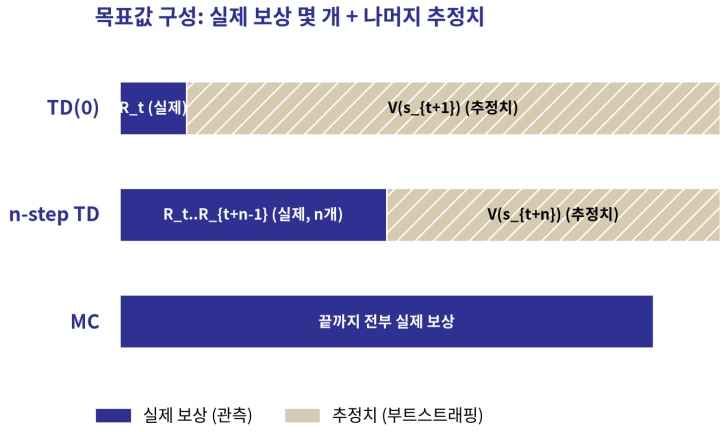
7.1 n-step TD: MC와 TD(0) 사이의 다이얼

복습: 두 극단 — MC와 TD(0)

- MC의 목표값: 에피소드 **끝까지의** 실제 보상 합(실제 리턴) G_t — 추정치 없음.
- TD(0)의 목표값: $r + \gamma V(s')$ — 실제 보상 1개 + 나머지 추정치.

n-step TD: n개의 실제 보상 + 나머지 추정치

그림으로: 세 방법의 목표값 구성



TD(0)은 실제 보상 1개, n-step TD는 n개, MC는 끝까지 전부. 나머지는 항상 **추정치(부트스트래핑)**로 채운다 — 셋은 같은 구조의 다른 길이일 뿐이다.

n-step 리턴의 정의

n-step 리턴 $G_t^{(n)}$

$$G_t^{(n)} = R_t + \gamma R_{t+1} + \cdots + \gamma^{n-1} R_{t+n-1} + \gamma^n V(s_{t+n})$$

- $n = 1$이면 **정확히 TD(0)**, $n \to \infty$(끝까지)이면 **정확히 MC**.
- n: 그 사이 어딘가를 상황에 맞게 고르는 **하이퍼파라미터**.

세 방법 한눈에 — 그리고 "갱신 시차"

	n	실제 보상	추정치(부트스트래핑)	갱신
TD(0)	1	R_t 1개	$V(s_{t+1})$	매 스텝 즉시
n-step TD	n	$R_t, \ldots, R_{t+n-1}$ n개	$V(s_{t+n})$	첫 갱신 $n-1$ 스텝 지연
MC	∞	끝까지 전부	없음	에피소드 종료 시

기다려야 하는 시간(학습의 시차)

$G_t^{(n)}$을 계산하려면 t에서 $t+n-1$까지의 보상이 **다 모여야** 하므로, t번 상태의 **첫** 갱신은 최소 $n-1$ 스텝 뒤에야 가능. n이 클수록 시차 $\uparrow$.

이 시차가 갱신 **횟수**에는 영향을 주지 않는다(반직관적 — FAQ에서 확인, 실습에서 직접 세어보자).

환경: 7개 상태 random walk

- 상태 0과 6: **터미널**, $1 \sim 5$: 이동 상태.
- 매 스텝 50:50 좌우 이동(무작위 정책).
- 왼쪽 끝 -1, 오른쪽 끝 $+1$ 보상. $\gamma = 0.9$, 상태 3에서 시작.

상태	참값 V^π
0	0 (터미널)
1	-0.5643
2	-0.2539
3	0 (시작, 대칭 중심)
4	$+0.2539$
5	$+0.5643$
6	0 (터미널)

좌우 미러링(대칭성)으로 $V(3) = 0, V(2) = -V(4), V(1) = -V(5)$.

참값의 모양

7-state random walk에서 $n = 3$으로 10 에피소드 ($\alpha = 0.1$, seed 고정) 학습하면, 참값의 모양을 대략 따라가며 **오른쪽 끝(+1 목표)에 가까울수록 가치가 매끄럽게 단조 증가**(상태 $1 \approx -0.48$ 에서 오른쪽으로, 상태 5 $\approx +0.37$까지).

에피소드가 10개뿐이라 참값까지 다 도달하진 않지만, **방향(오른쪽 > 왼쪽)은 이미 잡혀** 있는 상태.

n-step TD 코드 (1/2)

```python
def n_step_td(env_step, n, n_episodes, alpha, gamma,
              n_states, start_state):
    V = [0.0] * n_states
    for _ in range(n_episodes):
        states, rewards = [start_state], []
        s, T, t = start_state, float('inf'), 0
        while True:
            if t < T:
                a = random.randrange(2)   # 데모용무작위정책
                ns, r, done = env_step(s, a)
                states.append(ns); rewards.append(r)
                if done:
                    T = t + 1
                s = ns
            tau = t - n + 1       # 번tau 상태를지금갱신
            # --- 핵심줄은 3 다음슬라이드에서뜯어봄     ---
            if tau >= 0:
                G = n_step_target(tau)      # 목표값계산
                V[states[tau]] += alpha * (G - V[states[tau]])
            if tau == T - 1:
                break
            t += 1
    return V
```

환경 `env_step`: 0과 6이 터미널, `(next_state, reward, done)` 반환.

n-step TD 코드 (2/2): 핵심 3줄

```
            if tau >= 0:
                G = sum(gamma ** (i - tau) * rewards[i]
                        for i in range(tau, min(tau + n, len(rewards))))
                if tau + n < T:
                    G += gamma ** n * V[states[tau + n]]   # 추정치항
                V[states[tau]] += alpha * (G - V[states[tau]])
```

- $\tau \geq 0$: 실제 보상 n개가 다 모인 시점이므로 $G_\tau^{(n)}$ 계산 가능.
- `tau + n < T`: n 스텝 윈도우가 끝나기 **전에** 에피소드가 끝나지 않았다면 $\gamma^n V(s_{\tau+n})$을 추가. 끝나면 **추정치 항 생략**(절단 목표값).

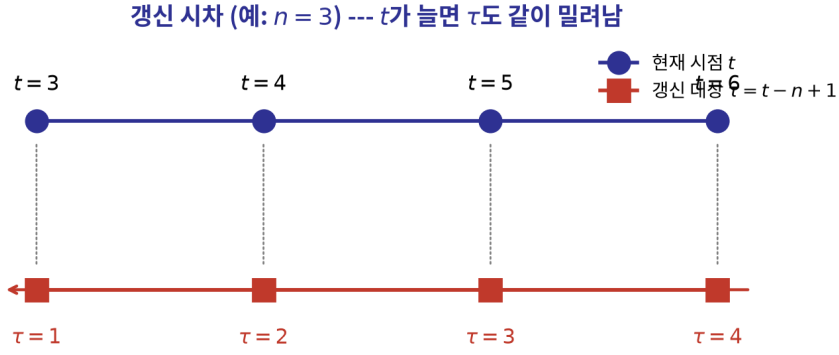
왜 $\tau = t - n + 1$인가 — 갱신 시차

- **갱신 시차 t - n + 1의 이유**

갱신 횟수는 변하지 않는다

에피소드가 T 이동 상태면 $n = 1$이든 $n = T + 1$이든 **모든 이동 상태마다 정확히 1번씩**(총 T번) 갱신된다. n이 바꾸는 것은 갱신 **횟수**가 아니라, 첫 갱신까지의 **시차**와 목표값의 **구성**(실제/추정 비율).

그림으로: 갱신 시차 $\tau = t - n + 1$



t가 한 칸씩 나아갈 때마다 갱신 대상 τ도 함께 한 칸씩 밀려 나온다 — **지연폭 $n-1$은 항상 고정**, 갱신되는 **시점**만 이동한다.

중요: 목표값의 할인 지수를 세어라

- $G_t^{(n)}$ 정의에서 R_{t+k}의 지수는 k

```
G = sum(gamma ** (i - tau - 1) * rewards[i] for i in ...)
# 각보상이  gamma만큼^{-1} 덜"" 할인된 -> 편향된목표값
```

- 지수 하나가 틀리면 편향된 목표값을 끝없이 쓰는 것

online 관점 vs forward 관점

online 관점 (online view)

- 학습이 **진행되면서** 목표를 만들어 갱신.
- "지금 t 스텝을 왔다" $\rightarrow$ 지금까지 **실제로 쌓인** $R_t, \ldots, R_{t+n-1}$으로 $G_t^{(n)}$ 계산.
- 위 코드의 `tau = t - n + 1`과 `tau + n < T` 분기가 바로 산물.
- 실제 알고리즘의 **실행 순서**(갱신 시차, 에피소드 끝 처리)를 정확히 반영.

forward 관점 (forward view)

- 같은 경험(trajectory) 위에서 n을 바꾸면 **서로 다른 목표값**이 생김.
- $n = 1$: $R_t + \gamma V(s_{t+1})$
- $n = 2$: $R_t + \gamma R_{t+1} + \gamma^2 V(s_{t+2})$
- "n이 커지면 MC에 가까워진다"는 직관(목표값 **정의 자체**에서 $n \rightarrow \infty$가 MC 리턴)을 가장 투명하게 보여줌.

두 관점은 **동일한 알고리즘**을 기술. 7.2의 적격흔적은 "모든 n을 동시에" 쓰면서 이 연결의 핵심이 됨.

왜 n이 커지면 편향은 줄고 분산은 커지는가

- $G_t^{(n)}$**의 기대값은 정확히** $V^\pi(s_t)$
- **학습 초반** $V \neq V^\pi$**: 편향의 출처**

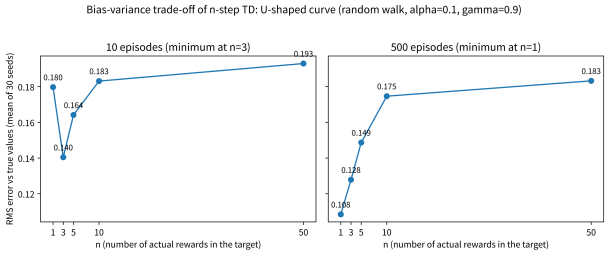
편향 $\downarrow$ n이 커지면 목표에서 추정치 비중 $\downarrow$, 실제 보상 비중 $\uparrow \Rightarrow V$가 부정확해도 목표값이 참값에 가까워짐.

분산 $\uparrow$ 실제 보상 합이 길어지면 각 스텝의 전이·보상 무작위성이 더 많이 누적됨. MC가 "분산 극대"인 이유와 정확히 같음.

U자 곡선: n별 RMS 오차 (30 seed 평균)

n	RMS 오차 (10 에피소드)
1 (TD(0))	0.4007
3	**0.3061** ← 이 환경·예산에서 최
5	0.3493
10	0.3994
50 (사실상 MC)	0.4170

설정: $\alpha = 0.1$, $\gamma = 0.9$, 10 에피소드, 30 seed.
오차가 n에 대해 **U자**.



(좌) 10 에피소드: 최저점 $n = 3$. (우) 500 에피소드: 최저점이 $n = 1$ 쪽으로 **이동** ($n = 1$: 0.268, $n = 3$: 0.336, $n = 50$: 0.457).

- $n = 1$**: 편향,** $n = 50$**: 분산**

결론: 보편적인 최적 n은 없다

핵심

최적 n은 에피소드 예산 · 환경의 확률적 정도 · 할인율에 따라 움직이는 하이퍼파라미터다.
이것이 실전에서 n을 검증으로 튜닝해야 하는 이유이며, 7.2의 적격흔적이 "n을 고르는 문제" 자체를 피하는(모든 n을 흔적으로 가중합) 대안인 이유이기도 하다.

n을 고르는 경험칙

- (i) 에피소드 **짧고** 경험 **많다**: $n = \infty$(MC)나 작은 n(TD(0))이 안전 — 편향을 씻어낼 데이터가 있으므로.
- (ii) 에피소드 **길고** 경험 **적다**: 중간 n(3∼10)이 편향을 줄이면서 분산을 관리.
- (iii) 에피소드 **끝이 없다**(continuing task): MC 자체가 불가능 ⇒ 유한 n 또는 적격흔적(7.2).

자주 하는 실수 (1/2)

- tau = t - n + 1이 정확

자주 하는 실수 (2/2)

- **“n > 에피소드 길이면 갱신 안 된다”는 오해.** 갱신은 모든 이동 상태마다 정확히 1번(꼬리 갱신으로 절단 목표값 처리). n이 바꾸는 것은 횟수가 **아니라** 시차와 목표값 구성.

자주 하는 실수 (3/3) + 실습

- **"이 환경에서 $n = 3$이 최적"을 일반화.** U자 최저점은 예산·환경에 따라 움직임(10 에피소드 $\to n = 3$, 500 에피소드 $\to n = 1$). 실전 표준: **검증 데이터로 n을 튜닝**.

실습: U자 곡선을 직접 만들어보기

1. 같은 random walk에서 $n \in \{1, 3, 5, 10, 50\}$, 10 에피소드 ($\alpha = 0.1$, $\gamma = 0.9$, 30 seed 평균) $\to$ RMS 비교. 최저점 $n = 3$, RMS 0.3061 재현.
2. **추가 과제**: 에피소드 10 $\to$ 500으로 늘려 다시 그리면 최저점 $n = 1$으로 이동. "실전에서 n을 고를 때 어떤 상황을 고려해야 하는가"를 한 문장으로 설명. (힌트: 예산 적을 때/많을 때 최저점 **위치**가 어떻게 달라지는가? 왜?)

확인 문제 (1/2)

1. $n = 1$**이 정확히 TD(0)임을 코드로 확인.** n=1로 실행하고 갱신에 쓰인 G가 $R_\tau + \gamma V(s_{\tau+1})$과 같은지 한 스텝 추적. (힌트: tau + 1 < T일 때만 추정치 항 추가. 마지막 스텝에서는 $G = R_\tau$만 됨 — "터미널 V는 0"과 일치하는가?)
2. $n \to \infty$**가 MC임을 확인.** 에피소드가 길이 T에서 끝나면($V(s_T) = 0$), $n \geq T$일 때 목표값이 실제 리턴 $G_t = \sum_{k=0}^{T-t-1} \gamma^k R_{t+k}$과 같은지 보여라.

확인 문제 (2/2)

3. **보상 가중치 버그의 효과.** `gamma ** (i - tau)`를 `(i - tau - 1)`로 바꾸고 $n = 1$, 2000 에피소드 $\rightarrow$ 참값 미도달 확인. “목표값의 편향이 **수렴 고정점 자체를 바꾼다**는 점을 어떻게 보여주는지” 한 문장.
4. **갱신 횟수 세기.** 에피소드 길이 T에서 $n = 1, n = 3, n = T + 1$ 각각이 **몇 번** 갱신되는지 코드 추적. (힌트: τ의 범위와 `break` 시점. 세 경우가 **모두 같은가?** 같은 이유?)

자주 묻는 질문 (1/2)

- **n-step TD도 부트스트래핑인가?** — 그렇다. $n < \infty$ 하면 목표값에 추정치 $V(s_{t+n})$이 들어 있으므로. $n \to \infty$(끝까지 실제 보상만)일 때만 순수 MC.
- **갱신 횟수는 TD(0)·MC와 어떻게 다른가?** — **같다**. T 이동 상태면 모두 T번. n-step은 첫 $n-1$ 상태가 "늦어지는" 뿐이지 사라지지 않음(꼬리 갱신까지). n이 바꾸는 것은 시차와 목표 구성. MC는 갱신 **시점**이 끝난 뒤라는 점만 다름.

자주 묻는 질문 (2/2)

- **실전에서는 n을 얼마로?** — 후보 4∼10개를 **검증 성능**으로 비교. 이 절의 실습이 그 튜닝의 축소판. DQN 구현 중엔 버퍼에서 미니배치를 이어붙여 n-step 목표값을 만드는 변형도 (Week 9 실습에서 마주침).
- **"n이 크면 분산 큼"은 갱신당 변동이 크다는 뜻인가?** — 그렇다. n 스텝치 전이·보상 무작위성이 합쳐짐(보상 0의 스텝도 "어디로 갔는가"의 전이 무작위성은 누적). 같은 총 정보를 작은 덩어리(잦은 갱신) vs 큰 덩어리(드문 큰 갱신)로 나누는 것 = ML1 Week 9의 **배치 크기 트레이드오프**와 같은 구조.
- **적격흔적(7.2)은 정말 같은 것을 계산하나?** — "같은 효과(모든 n 가중합)를 **다른 순서**로 만든다"가 정확.

7.2 적격흔적과 파라미터 튜닝 실습

7.2의 두 가지

- 둘째, n 하나를 고르는 대신 **모든 n을 동시에** 처리하는 **적격흔적(eligibility trace)**:
- "이 상태가 **얼마나 자주**, **얼마나 최근에** 방문됐는가"를 기록한 흔적 $e(s)$로, TD 오차가 발생하면 **흔적 크기에 비례해 모든 상태를 동시에** 갱신.

이 절의 주제 하나

편향-분산 트레이드오프(ML1 Week 4.1)를 "구체적인 숫자"로 만나는 방법.

실습: n을 바꿔가며 최적 지점 찾기

- 같은 7-state random walk 환경. 참고: 이 절의 설정에서는 **참값** $V(s) = s/6$(예: $V(3) = 0.5$)이 정확히 알려져 있음.
- `n_step_td`로 10 에피소드 학습, n별 RMS 오차, **20회 반복 평균**:

n	평균 RMS 오차
1	0.3410
3	0.1830
5	**0.1687** ← 최적
10	0.1883
50	0.2178

- 다시 **U자** — 양 극단($n = 1, n = 50$)보다 중간 ($n = 5$)이 최저. (7.1의 30 seed 실험에선 $n = 3$이 최저였음 — 설정·시드 수에 따라 바닥 위치가 미세하게 움직임.)
- 이 표가 아래 그림의 **왼쪽 패널**을 그대로 그린 것.

적격흔적: 모든 n을 동시에

TD(λ)의 두 갱신식

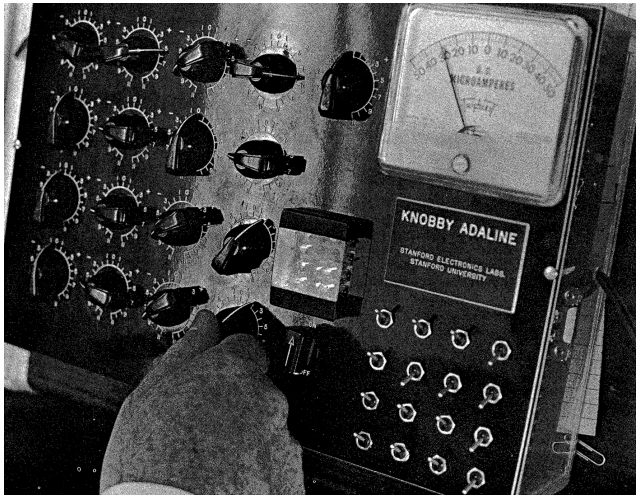
$$e(s) \leftarrow \gamma\lambda\, e(s) + \mathbb{1}[s = s_t], \qquad V(s) \leftarrow V(s) + \alpha\, \delta_t\, e(s) \;\; (\text{모든 } s\text{에 대해})$$

- $\lambda = 0$ **TD(0),** $\lambda = 1$ **MC**
- n**-step TD와 적격흔적의 등가성**

기원: ADALINE의 “자격 있는 시냅스”

- ADALINE(최소제곱학습기)에 “그 시점에 학습에 **자격(eligibility)**이 있는 시냅스가 어디인가”를 기록하는 장치를 처음 도입.
- “어떤 **상태**가 지금 TD 오차에 기여할 자격이 있는가” 로 옮겨온 것이 지금의 적격흔적.

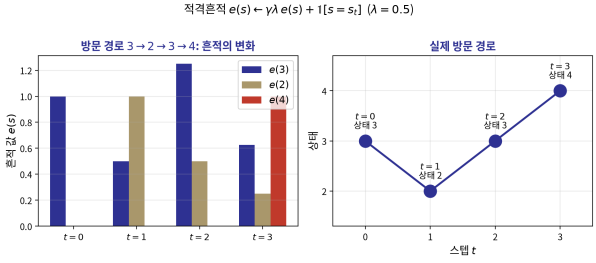
“자격을 남기고, 나중에 그 자격만큼 크레딧을 배분한다”.



“Knobby ADALINE” — Stanford Electronics Labs, Stanford University, 1963
손잡이(rheostat) 하나하나가 조정 가능한 시냅스 가중치

손으로 추적: $3 \to 2 \to 3 \to 4$

- **갱신 순서: 감쇠, $+1$, V 갱신**



추적 표 (1/2): $t = 0 \sim 2$

스텝 t	현재 상태	δ_t	이번 스텝 V 갱신
0	3	$+0.4$	$e(3) = 1$: $V(3) \mathrel{+}= 0.1 \cdot 0.4 \cdot 1 = +0.040$
1	2	$+0.3$	$e(2) = 1, e(3) = 0.5$: $V(2) \mathrel{+}= +0.030$, $V(3) \mathrel{+}= 0.1 \cdot 0.3 \cdot 0.5 = +0.015$
2	3	$+0.5$	$e(2) = 0.5, e(3) = 1.25$: $V(2) \mathrel{+}= +0.025$, $V(3) \mathrel{+}= 0.1 \cdot 0.5 \cdot 1.25 = +0.0625$

- $t = 1$: $t = 0$의 $e(3) = 1$이 0.5로 **감쇠**, 현재 2에 $+1$.
- $t = 2$: $e(3) = 0.5 \cdot 0.5 = 0.25$로 감쇠한 뒤 **재방문**에 $+1 \Rightarrow e(3) = \mathbf{1.25}$ — **방문 빈도가 흔적에 누적**.

추적 표 (2/2): $t = 3$에서 한 오차, 세 상태

스텝 t	현재 상태	δ_t	이번 스텝 V 갱신
3	4	$+0.2$	$e(2) = 0.25, e(3) = 0.625, e(4) = 1$: $V(2)$ += $+0.005$, $V(3)$ += $+0.0125$, $V(4)$ += $0.1 \cdot 0.$

- **TD 오차 0.2가 세 상태(2, 3, 4)를 동시에 갱신**
  - 현재 상태 4(흔적 1)가 가장 크게,
  - 직전 상태 3(0.625)이 그다음,
  - 한 번 더 지난 상태 2(0.25)가 가장 작게.

  이것이 "**후방으로 크레딧을 퍼뜨린다**"는 뜻.

λ가 n을 어떻게 "대체"하는가

- n-step TD의 목표값 $G_t^{(n)}$: **단 하나의** n-step 리턴(정확히 n 스텝만 보고 나머지는 추정치).
- 적격흔적: 흔적 감쇠율 $\gamma\lambda$ 덕분에 t 스텝 전에 방문한 상태의 흔적은 대략 $(\gamma\lambda)^t$로 감소.

TD(λ)의 목표 = 한 줄

TD(λ)의 목표 $\approx$ "모든 n-step 목표의 가중평균, n에 대한 가중치 $\propto \lambda^{n-1}$"

“하나를 고르는 문제”에서 “분포의 형태”로

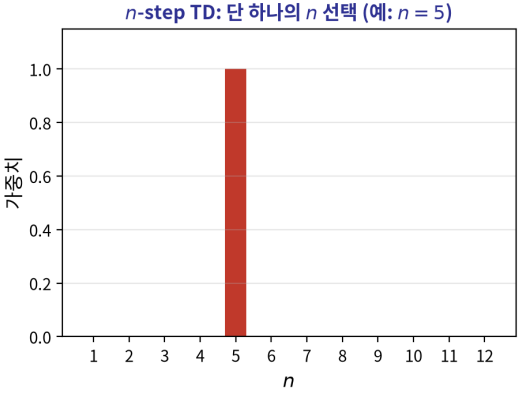
- n-step이 **“하나의 n을 고르는”** 문제였다면, TD(λ)는 **“가중치 분포 λ^{n-1}의 형태”**를 λ 하나로 조절하는 문제.
- 같은 편향-분산 트레이드오프를, **하나를 고르는 문제에서 분포의 형태를 고르는 문제로 승화**시킨 것.

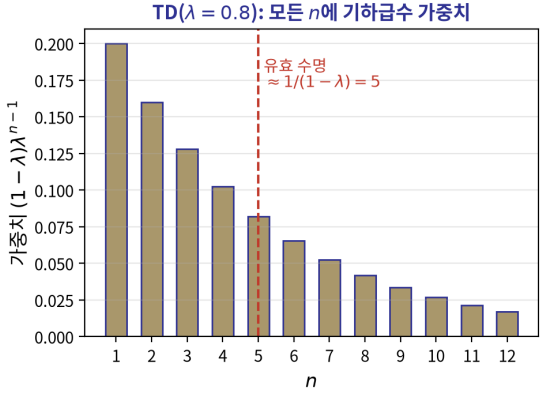
직접 확인해야 할 한 줄

λ를 n으로 환산하는 근사 대응: TD(λ)의 **유효 수명(effective horizon)** $\sim 1/(1-\lambda)$ **스텝**. $\lambda = 0.8$이면 대략 $1/(1-0.8) = 5$ 스텝 전후 — 이번 실험에서 $n = 5$가 최적, $\lambda = 0.8$이 최적인 것과 **동일한 방향**. 정확한 1:1 변환은 아니지만, “$\lambda = 0.8$이 5스텝 정도 내다보는 것과 비슷하다”는 직관은 잡아둠.

그림으로: 가중치 분포의 형태

``하나를 고르는 문제(왼쪽) vs ``분포의 형태를 고르는 문제(오른쪽)





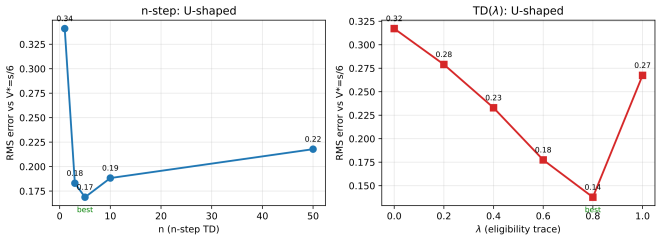
실습: λ를 바꿔가며 최적 지점 찾기

- λ도 하이퍼파라미터 $\Rightarrow$ n과 똑같이 **데이터로 고름**.
- 같은 random walk, 10 에피소드, $\alpha = 0.2$, $\gamma = 1$, λ 0∼1, **20회 반복 평균** RMS:

λ	평균 RMS 오차
0.0 (TD(0)와 동일)	0.3172
0.2	0.2790
0.4	0.2329
0.6	0.1774
0.8	**0.1377** ← 최적
1.0 (사실상 MC)	0.2674

- 다시 **U자**. 좌측 $\lambda = 0$은 정확히 TD(0)($n = 1$, $\alpha = 0.2$ 결과와 같은 0.3172)라 편향 씻어내지 못함. 우측 $\lambda = 1$은 MC에 가까워 분산 큼.
- **"$\lambda = 0.8$이 만능 정답"이 아니다** — 환경·에피소드 수(예: 1000)가 바뀌면 U자 바닥은 **완전히 다른 위치**로 이동. 에피소드가 충분히 많으면 편향이 씻겨 λ를 1에 가깝게(MC 쪽) 가져가도 좋은 경우 많음.
- **"데이터로 고르는 것" 자체가 방법 — 검증 데이터·반복 실험으로 매번 재확인하는 습관이 실전의 본체.**

두 개의 U자 나란히



(좌) n-step: 최저 $n = 5$, 0.17. (우) TD(λ): 최저 $\lambda = 0.8$, 0.14. 10 에피소드 · $\alpha = 0.2$ · 20회 반복 평균.

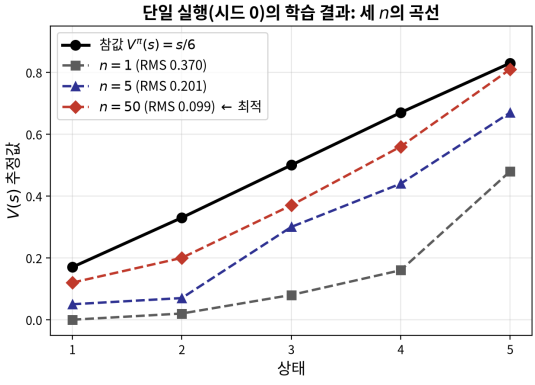
두 패널이 나란하니 메시지가 선명하다

n을 고르는 일과 λ를 고르는 일은 같은 편향-분산 트레이드오프의 두 모습. 둘 다 U자. "$n = 5$"와 "$\lambda = 0.8$"은 서로 다른 문제를 푸는 게 아니라, **"같은 U자에서 지금 이 데이터양·이 문제에서 바닥이 어디인지"를 각각의 다이얼 위에서 재확인한 결과**일 뿐.

단 한 번의 실행은 믿지 마라

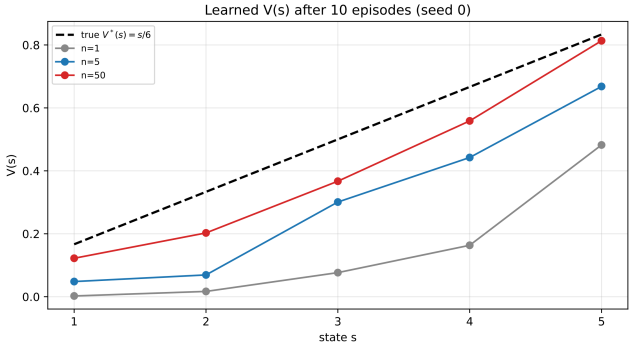
	$V(1 \sim 5)$ (RMS)
$n = 1$	0.00 0.02 0.08 0.16 0.48 (0.370)
$n = 5$	0.05 0.07 0.30 0.44 0.67 (0.201)
$n = 50$	0.12 0.20 0.37 0.56 0.81 **(0.099)**
참값	0.17 0.33 0.50 0.67 0.83

$n = 50$**이 이번엔 제일 좋다** ← 단일 실행은 전체 U자를 **역전**시킬 수 있다.



단일 실행(시드 0)의 세 곡선 — $n = 50$(빨강)이 이번 실행에선 참값에 가장 가까움.

단일 시드 vs 20회 평균



단일 실행(시드 0, $\alpha = 0.2$, 10 에피소드)의 학습 $V(s)$와 참값 $V^* = s/6$. $n = 1$(회색)은 거의 0에서 움직이지 못해 큰 편향, $n = 5$(파랑)은 중간, $n = 50$(빨강)은 이번 실행에선 참값에 가장 가까움.

함정의 본질

하이퍼파라미터를 고르면서 단일 실행(단일 시드, 단일 검증집합)에 의존하면, "U자를 찾겠다"는 전제가 완전히 무너진다.

최적 지점을 찾는 실험은 반드시 여러 시드/반복으로 평균내야 한다 — 이 절이 실제로 강조하는 튜닝의 전제 조건.

이 절의 하이퍼파라미터 지도

방법	목표값	하이퍼파라미터(다이얼)	이 절의 결과
TD(0)	$r + \gamma V(s')$	—($n = 1$)	U의 좌측 극단
n-step TD	n 스텝 실제 + 나머지 추정	**n**	U의 바닥 $n \approx 5$
TD(λ)	모든 n의 가중평균	**λ**	U의 바닥 $\lambda \approx 0.8$
(참고) MC	실제 리턴 G_t	—($n = \infty, \lambda = 1$)	U의 우측 극단

- **α는 U자 깊이, 에피소드 수는 바닥 위치**

자주 하는 실수 (1/2)

- 편향과 분산을 합친 총 오차(RMS)로 봐야 함

자주 하는 실수 (2/2)

- $\lambda = 0$이 TD(0)와 일치하는지 확인

확인 문제

1. **왜 U자인가.** 좌측 극단(짧은 부트스트래핑) = **편향**의 문제, 우측 극단(긴 부트스트래핑) = **분산**의 문제. 그리고 "에피소드 1000으로 늘리면 U자 바닥이 어느 쪽으로 이동할지, 왜" 한 문장씩.
2. **흔적의 두 역할.** $t = 3$ 행에서 상태 4/3/2의 갱신 크기가 각각 다른 이유를 **흔적 값(1, 0.625, 0.25)**을 들어 설명. 상태 3이 상태 2보다 **더 큰 누적 갱신**을 받은 **두 번째 이유**(감쇠와 **무관**한 이유) 짚기. (힌트: 재방문 $\Rightarrow e(3) = 1.25$.)
3. **코딩.** `td_lambda`($\lambda = 0$)의 마지막 V가 같은 설정 `n_step_td`(n=1)과 RMS가 일치하는지 ($< 10^{-9}$ 차이) `assert`로 확인. 이 검증이 "적격흔적 코드가 올바른가"를 어떻게 보장하는지 한 문장.

자주 묻는 질문 (1/2)

- **에피소드 수가 많으면 $\lambda = 1$(MC) 쪽이 좋아지나요?** — MC의 단점은 **분산**이지 편향이 아님. 에피소드 10개면 각 갱신이 "그 에피소드의 운"에 크게 흔들리지만 1000개면 무작위성이 **평균에 의해 서로 상쇄**되어 MC의 분산이 거의 사라짐 $\Rightarrow$ 남는 건 편향뿐(MC는 0) $\Rightarrow$ $\lambda = 1$이 가장 정확. 반대로 에피소드가 적으면 편향·분산 중 **어느 쪽이 더 큰 손해인가**가 U자 좌/우 기울기를 정함.

자주 묻는 질문 (2/2)

- **"모든 상태 동시 갱신"의 계산 비용은 터지나?** — 갱신 대상 = **방문한 상태 수**(보통 5개). 매 스텝 5번 곱셈+덧셈 = TD(0)의 "매 스텝 1번" 대비 **상당히 가벼운 증가**. 상태 10만 개(고차원 격자) 수준에선 "모든 상태 순회"가 병목 $\Rightarrow$ 실전에서는 **흔적 0인 상태를 건너뛰는 희소(sparse) 구현**, 또는 DQN처럼 **신경망 + 적격흔적 조합**. 핵심 구조적 아이디어(후방 크레딧 배분)는 그대로, 계산 표현만 상황에 맞게 최적화.

이 절의 핵심 한 줄

적격흔적은 "n 하나를 고르는 것"을 "모든 n의 가중분포를 λ 하나로 조절하는 것"으로 승화시킨 기법. 분포의 형태(λ) + 학습률(α) + 데이터량(에피소드 수) 셋이 **함께** U자 바닥을 정하므로, 이 셋을 **데이터로 튜닝**하는 것이 실전의 전부.

7.3 계획과 학습: Dyna-Q

경험을 버리지 않는 법

- 그런데 한 스텝 $(s, a) \to (r, s')$에는 두 가지가 담겨 있다:
  - (1) **Q를 갱신할 신호**(TD(0) 갱신)
  - (2) **환경에 대한 지식** — "(s, a)를 하면 대략 (r, s')이 나온다".

  모델-프리는 (2)를 **버림**.

(Sutton, 1990 박사논문)

이 정보를 간단한 **모델**로 저장해뒀다가, 실제 상호작용 사이사이에 **가상의 추가 학습(계획, planning)**을 끼워 넣는다.



Richard S. Sutton — Dyna 아키텍처 제안(1990, PhD 논문), Reinforcement Learning: An Introduction 공저자

2025년 NeurIPS(샌디에이고) 강연 모습

왜 모델을 저장하는가 — 상호작용이 비쌀 때

- 실제 환경 상호작용의 **비용이 비쌀수록**, “간접적으로 값을 아는 것”의 가치가 큼.
- 실제 스텝 1회당 Q 갱신 횟수를 10배로 늘려주는 것은 **학습에 소요되는 실세계 시간을 10분의 1로 줄여주는** 것과 같음.

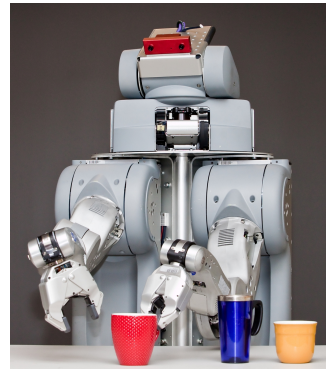
Dyna-Q의 계산

learning rate를 높이는 대신, **같은 실제 경험**으로 더 많은 갱신을 만들어내는 것. 아주 단순한 구조로 해냄: **“본 (s, a)의 마지막 (r, s')를 기억하는 표”**.



NASA 큐리오시티 화성 탐사선 자가촬영 (2015)

지구↔화성 통신 지연 평균 **14분 6초**



Willow Garage PR2 로봇의 물체 조작 실험 (2009)

로봇 팔의 한 번의 시도 = 초 단위 물리 동작 + 마모 비용

Dyna-Q 코드: 경험 → 학습 → 모델 저장 → 계획

```python
def dyna_q(env_step, n_states, n_actions, n_episodes,
           planning_steps, alpha, gamma, epsilon, start_state):
    Q = [[0.0] * n_actions for _ in range(n_states)]
    model = {}  # (s, a) -> (r, s') 관찰한것을그대로저장
    for _ in range(n_episodes):
        s = start_state
        for _ in range(100):
            a = random.randrange(n_actions) if random.random() < epsilon \
                else max(range(n_actions), key=lambda x: Q[s][x])
            ns, r, done = env_step(s, a)
            Q[s][a] += alpha * (r + gamma * max(Q[ns]) - Q[s][a])  # 1) 실제학습
            model[(s, a)] = (r, ns)                                 # 2) 모델갱신
            for _ in range(planning_steps):                         # 3) 계획
                (ps, pa), (pr, pns) = random.choice(list(model.items()))
                Q[ps][pa] += alpha * (pr + gamma * max(Q[pns]) - Q[ps][pa])
            s = ns
            if done:
                break
    return Q
```

실제 숫자: 같은 환경, 계획이 있는가

- 같은 격자 환경에서, **목표 상태(state 3, 오른쪽으로 가는 행동)의 Q값이 0.5를 넘어서기까지**의 에피소드 수:

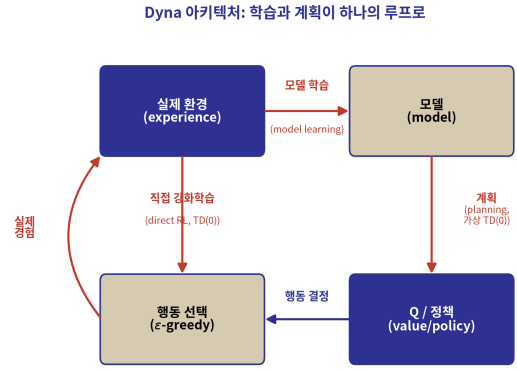
설정	0.5 초과까지 에피소드
`planning_steps=0` (순수 Q-learning)	253
`planning_steps=10`	**9** ← 매 실제 스텝마다 모델로 10번 추가 학습

핵심

같은 실제 경험으로, 계획이 있는 쪽이 훨씬 더 많은 것을 뽑아낸다.

역사적 맥락: 학습과 계획을 하나의 루프로

- 학습과 계획을 하나의 루프 안에
- Dyna의 골격: 실제 경험 모델로 계획



손으로 계획 한 스텝: 3상태 예제

- 상태 S(출발) $\to X \to T$(목표, 종료), 각 상태마다 행동 하나, 전이는 **결정론적**.

$$S \xrightarrow{a} X \text{ (보상 } 0), \qquad X \xrightarrow{a} T \text{ (보상 } +1), \qquad \gamma = 0.9,\ \alpha = 0.5$$

- 참값(벨만방정식 **거꾸로** 쓰기): $Q(T) = 0$, $Q(X) = 1 + \gamma \cdot 0 = 1$, $Q(S) = 0 + \gamma \cdot 1 = 0.9$.

추적 (1/3): 실제 경험만 있으면 $Q(S)$는 안 움직임

- 초기 $Q(S) = Q(X) = Q(T) = 0$.
- $Q(S)$**는 안 움직임**

추적 (2/3): 계획 한 스텝이 $Q(S)$를 움직인다

- 모델이 예측한 (r, s')을 그대로 TD(0) 갱신에 사용:

$$Q(S) \leftarrow 0 + 0.5\Big(0 + 0.9 \cdot \underbrace{Q(X)}_{0.5} - 0\Big) = 0.5 \times 0.45 = 0.225$$

Dyna-Q의 본질

에이전트는 S에 다시 발을 디딤 없이, X에서 막 배운 값(0.5)이 모델을 타고 S로 "거꾸로" 퍼졌다. 실제 경험이 한 상태의 Q를 바꾸면, 계획은 그 변화가 **모델로 연결된 다른 상태들**의 Q에까지 퍼지도록 만든다.

추적 (3/3): 몇 번만 더 하면 참값에 수렴

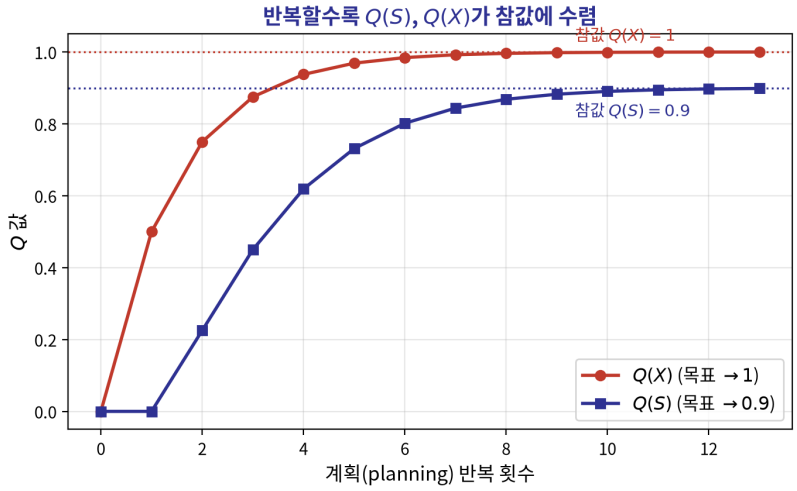
- 계획을 몇 번만 더 하면 $Q(S) \to 0.9$, $Q(X) \to 1$. (실제로 (X, a)가 뽑히면 $Q(X)$ 쪽이 1로, (S, a)가 뽑히면 $Q(S)$ 쪽이 0.9로 각각 접근.)

핵심 직관

TD(0)은 "현재 상태에서 **앞으로** 한 스텝"을 보지만, 계획은 "모델이 기억하는 (s, a)에서 **모델을 따라** 한 스텝"을 보므로, **실제 이동 없이도 상태 공간 전체에 걸쳐 값을 퍼뜨릴** 수 있다.

실제 경험 = 신뢰할 수 있는 신호, **계획** = 그 신호를 넓게 퍼뜨리는 역할.

그림으로: $Q(S)$, $Q(X)$의 수렴



세 가지 축을 한자리에

방법	목표값	특징
MC (Week 5)	실제 리턴 G_t	편향 없음, 분산 큼, 에피소드 종료 필요
TD(0) (Week 6)	$r + \gamma V(s')$	편향 있음, 분산 작음, 매 스텝 학습
n-step TD	n 스텝 실제 + 나머지 추정	둘 사이 절충, n이 다이얼
TD(λ)	모든 n을 흔적으로 가중평균	n 고정 불필요, λ가 다이얼
Dyna-Q	TD(0) + 학습된 모델로 계획	모델-프리와 모델-기반 결합

Block A 전체의 그림

이번 학기 Block A 전체가 사실 **"부트스트래핑을 얼마나 할 것인가"라는 하나의 질문에 대한 서로 다른 답**. 이번 장(7.1∼7.3)은 그 질문에 n**,** λ**, 계획**이라는 세 종류의 추가적인 다이얼/루프를 더했다.

실습: 5×5 격자의 환경과 참값

- **환경**: 5×5 격자. 출발 $(4,0)$(아래 왼쪽), 목표 $(0,4)$(위 오른쪽), 4방향 이동, 벽 부딪히면 자리에 남음. 매 이동 **보상** -1, 목표 도착 시 0으로 종료. $\gamma = 1$(유한하고 반드시 끝나는 과업이라 할인 불필요 — 5.2절 블랙잭과 같은 이유), $\alpha = 0.1$, $\varepsilon = 0.1$.

참값은 손으로 나온다

맨해튼 거리 $|4-0| + |0-4| = \mathbf{8}$ 스텝. 8번 이동 중 7번이 -1, 마지막(목표 도착)이 0 $\Rightarrow$ **최적 리턴 정확히** -7, $V^*(\text{출발}) = -7$.

"위 또는 오른쪽, $Q^* = -7$**"** — 위·오른쪽이 최단 경로 첫 스텝. 아래/왼쪽으로 한 칸 빠지면 9스텝이 되어 $Q^* = -8$. 학습 결과가 이 값에 접근하는지로 "잘 학습했는가"를 **숫자로 바로 판정**.

실습 코드: 5×5 격자 Dyna-Q

```
ACTIONS = [(-1, 0), (1, 0), (0, -1), (0, 1)]  # up, down, left, right
START, GOAL = (4, 0), (0, 4)

def step(s, a):
    r, c = s
    nr = min(max(r + ACTIONS[a][0], 0), 4)
    nc = min(max(c + ACTIONS[a][1], 0), 4)
    ns = (nr, nc)
    if ns == GOAL:
        return ns, 0.0, True
    return ns, -1.0, False

def dyna_q_grid(n_episodes, planning_steps,
                alpha=0.1, gamma=1.0, epsilon=0.1, seed=0):
    rng = random.Random(seed)
    Q, model, returns = {}, {}, []
    for _ in range(n_episodes):
        s, ret = START, 0.0
        for _ in range(100):
            a = rng.randrange(4) if rng.random() < epsilon \
                else max(range(4), key=lambda x: Q.get((s, x), 0.0))
            ns, r, done = step(s, a)
            Q[(s, a)] = Q.get((s, a), 0.0) + alpha * (
                r + gamma * max(Q.get((ns, x), 0.0) for x in range(4))
                - Q.get((s, a), 0.0))                # 1) 실제
            model[(s, a)] = (r, ns)                   # 2) 모델
            for _ in range(planning_steps):           # 3) 계획
                (ps, pa), (pr, pns) = rng.choice(list(model.items()))
                Q[(ps, pa)] = Q.get((ps, pa), 0.0) + alpha * (
                    pr + gamma * max(Q.get((pns, x), 0.0) for x in range(4))
                    - Q.get((ps, pa), 0.0))
            s, ret = ns, ret + r
            if done:
                break
        returns.append(ret)
    return Q, returns
```

dyna_q의 격자 버전. 실제/모델/계획 3단계가 같은 Q를 갱신.

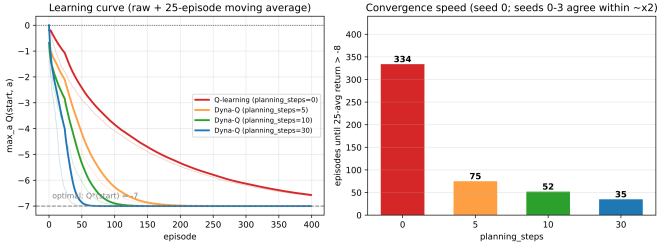
실험: planning_steps 0 → 30

- 지표: **"25에피소드 평균 리턴이 최적치(-7)에서 1.0 이내(> -8)로 처음 들어가는 에피소드"**.

planning_steps	수렴 에피소드 (시드 0)	시드 1∼3	400 에피소드 후 max Q(start)
0 (순수 Q-learning)	334	334∼387	-6.61
5	75	65∼75	-7.00
10	52	51∼55	-7.00
30	35	35∼61	-7.00

모델 커버리지: 400 에피소드 후 100개 (s, a) 중 95∼96개 모델에 저장.

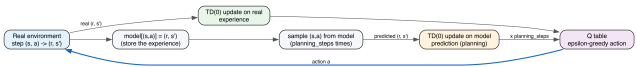
그래프로 보는 세 가지 발견



(좌) 매 에피소드 $\max_a Q(\text{출발}, a)$ 학습 곡선(원본 + 25에피소드 이동평균). 4개 곡선이 참값 -7에 접근. (우) "-8 위로 처음 오르는 에피소드" 막대 그래프 — 334(0) → 35(30), **약 9.5배** 빠름.

1. **가속은 극적.** 334/35 ≈ **9.5배**. 실제 상호작용 **35회**로, 계획 없이는 334회 걸렸던 학습이 끝.
2. **한계 효과 감소.** 0→10: 334→52(약 6배). 10→30: 52→35(약 1.5배)로만.
3. **모델은 거의 포화.** 100개 (s, a) 중 95∼96개 커버. 작은 격자라 거의 모든 (s, a)가 1회 이상 방문 — 이 커버리지가 계획 품질의 상한선.

Dyna-Q 루프 구조도



Dyna-Q 루프: (1) 실제 경험으로 TD(0) 갱신, (2) `model[(s,a)] = (r,s')` 저장, (3) 모델에서 `planning_steps`번 무작위 (s, a) 뽑아 가상 TD(0) 갱신. 실제/계획 갱신 **모두 하나의 Q 테이블**로 모이고, 그 Q에서 ε-greedy 행동을 뽑아 다시 환경으로.

왜 `planning_steps`이 하이퍼파라미터인지

7.2에서 n을 "편향-분산 사이 어딘가"로 튜닝했듯이, 여기서는 `planning_steps`를 **"계산 비용 대 가속 효과" 사이 어딘가**로 튜닝.

이번 실험: **10∼30이 좋은 구간**. 0(계획 없음)과 "무한히 크게"(아래 실수 3 참조)는 둘 다 극단.

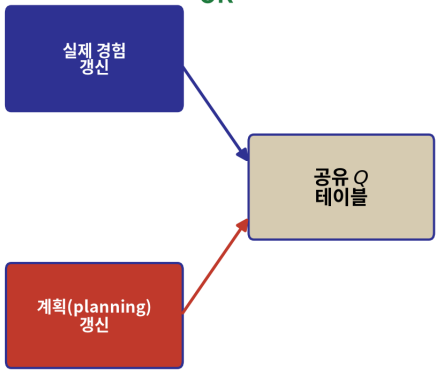
자주 하는 실수 (1/3)

- 희소 딕셔너리 직접 조회 시 KeyError
- model.items()에서 무작위 선택

자주 하는 실수 (2/3)

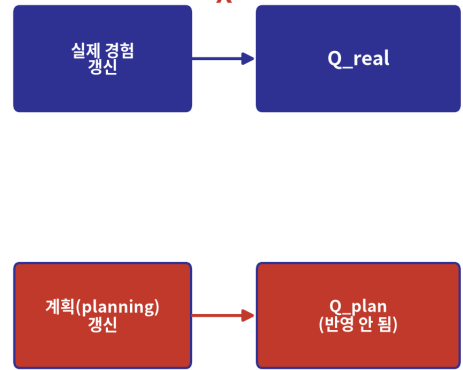
올바름: 하나의 Q를 공유

OK



실수: 두 Q로 분리

X



Dyna-Q: 두 갱신이 같은 Q 테이블을 공유해야 한다

자주 하는 실수 (3/3)

- **3. `planning_steps`를 아주 크게 잡기.** 계획 1회 비용 $\approx$ 실제 스텝 1회 비용.

실전

`planning_steps`도 α, ε과 **같은 하이퍼파라미터**로, **검증 성능으로 튜닝**.

확인 문제

1. **공유 테이블.** `dyna_q_grid`에서 계획 갱신(3번 단계)을 `Q_plan` 별도 딕셔너리로 분리하면, `planning_steps=30`의 수렴 에피소드가 순수 Q-learning(334)과 어떻게 달라지는지 **숫자 없이** 한 문장. (힌트: 계획의 산출물이 어디로 가는가.)
2. **모델 커버리지.** 400 에피소드 후에도 100개 (s, a) 중 4개(4%)가 모델에 안 들어감. 이 "없는 4개"가 계획에 미치는 영향을 `random.choice(list(model.items()))`의 관점에서 설명.
3. **참값 판정.** 학습 후 Q에서 $\max(Q[(\text{START}, 0 \sim 3)]) = -7.00$이고 arg max가 "위"(action 0) 또는 "오른쪽"(action 3)이면, 이 정책이 **최적**인 근거가 손계산 참값(위/오른쪽 $= -7$, 아래/왼쪽 $= -8$)에 어떻게 대응하는지 설명.

자주 묻는 질문 (1/2)

- **Dyna-Q의 모델은 항상 정확한가?** — 학습 초반엔 **전혀 그렇지 않음**. 몇 번밖에 못 가본 (s, a)쌍의 모델은 부정확하거나 아예 없을 수 있음. 그런데도 작동하는 이유: 모델이 부정확해도 **"그 부정확한 모델로 계획한 결과"가 "아무것도 안 하는 것"보다는 대체로 낫고**, 실제 경험이 쌓일수록 모델도 함께 정확해져 계획 품질도 좋아짐.
- **`planning_steps`를 무한히 늘리면 항상 좋아지나?** — 실제 스텝 1회당 계획에 쓰는 계산량이 커질 뿐, 계획 자체는 **이미 모은 경험 이상의 새 정보를 만들어내지 못함**. 모델이 부정확한 상태에서 계획만 과도하게 반복하면 그 부정확함이 Q에 더 강하게 반영 $\Rightarrow$ 오히려 해로울 수 있음(모델 오차의 "복리 효과"). 실전: **검증 성능으로 튜닝**.

자주 묻는 질문 (2/2)

- **Dyna-Q는 on-policy인가, off-policy인가?** — **off-policy**. Q 갱신 방식은 6.3절 Q-learning과 **완전히 같음** — 목표값에 $\max_a Q(s', a)$(탐욕). 계획이 모델에서 (s, a)를 무작위로 뽑는 것은 "행동 정책"이 아니라 "어떤 (s, a)의 Q를 갱신할지"를 고르는 것. 갱신 본질엔 여전히 max가 들어감.
- **DQN의 경험 재사용(experience replay, Week 9)과 무엇이 다른가?** — 공통: "과거 경험을 재사용해 실제 스텝당 갱신을 여러 번". 차이: **재사용의 원천**.
  - **재생**: 실제 (s, a, r, s') 튜플을 버퍼에서 다시 뽑아 씀 $\Rightarrow$ **모델 오차 없음**. 단, 경험한 전이만 재사용할 뿐 새로운 전이는 못 만듦.
  - **Dyna-Q 계획**: 모델 예측 (r, s')으로 갱신 $\Rightarrow$ **모델 오차 동반**. 단, 모델이 연결한 아직 경험하지 않은 상태의 전이까지 만들 수 있음.

  "안전하지만 새것을 못 만든다"(재생) vs "새것을 만들 수 있지만 오차가 동반된다"(계획) — 이 트레이드오프가 Week 9 이후 모델 기반 DQN 변형들의 줄거리.

언제 Dyna-Q가 특히 효과적인가

Dyna-Q가 좋은 상황

- 실제 상호작용이 **비싸고** (로봇, 물리 실험, 지연 큰 시스템)
- 환경이 충분히 **규칙적**해서 간단한 모델로 대략 잡아낼 수 있을 때.

모델-프리가 더 좋은 상황

- 상호작용이 **아주 싸고**(시뮬레이션, 게임)
- 모델 학습이 **어렵거나**(강한 난수성, 비마르코프성) 신뢰할 수 없을 때.

첫 번째 판단 기준

"모델을 만들 수 있는가, 만들어도 신뢰할 만한가" — Dyna-Q를 쓸지 말지의 첫 번째 기준.

이번 주차 정리

- **7.1 n-step TD** — $G_t^{(n)}$은 MC와 TD(0) 사이의 하나의 다이얼. `tau = t - n + 1`의 갱신 시차, 할인 지수 하나만 틀리면 $n = 1$이어도 **편향된 고정점**. 7-state random walk에서 30 seed 평균 $\Rightarrow$ **U자**. 보편적 최적 n은 없으므로 검증으로 튜닝.
- **7.2 적격흔적** — 흔적 $e(s)$로 하나의 TD 오차가 **모든 상태를 동시에** 후방 갱신. λ는 "n 하나 고르기"를 "가중치 분포 λ^{n-1}의 형태 고르기"로 승화. **단일 실행은 믿지 말 것** (20회 반복 평균). $\lambda = 0 \Leftrightarrow$ TD(0), $\lambda = 1 \Leftrightarrow$ MC 건성 체크.
- **7.3 Dyna-Q** — 본 (s, a)의 (r, s')를 모델로 저장, 실제 스텝마다 모델에서 가상 TD(0) 갱신. **실제/계획이 같은 Q 테이블 공유**가 전부. 5×5 격자: 334 → 35 에피소드(약 9.5배), 한계 효과 + 모델 커버리지가 상한.

다음 주 — Week 8. Block A 캡스톤

Gymnasium의 표 기반 환경에서 **n-step TD · TD(λ) · Dyna-Q를 팀 프로젝트로 직접 구현·비교**. 이번 주 세 다이얼이 **같은 질문("부트스트래핑을 얼마나 할 것인가")**의 서로 다른 답이라는 것만 기억하면, 비교 설계가 훨씬 수월해진다.

이미지 출처

- “Knobby ADALINE” 사진 (1963) — Stanford Today, Stanford Electronics Labs